

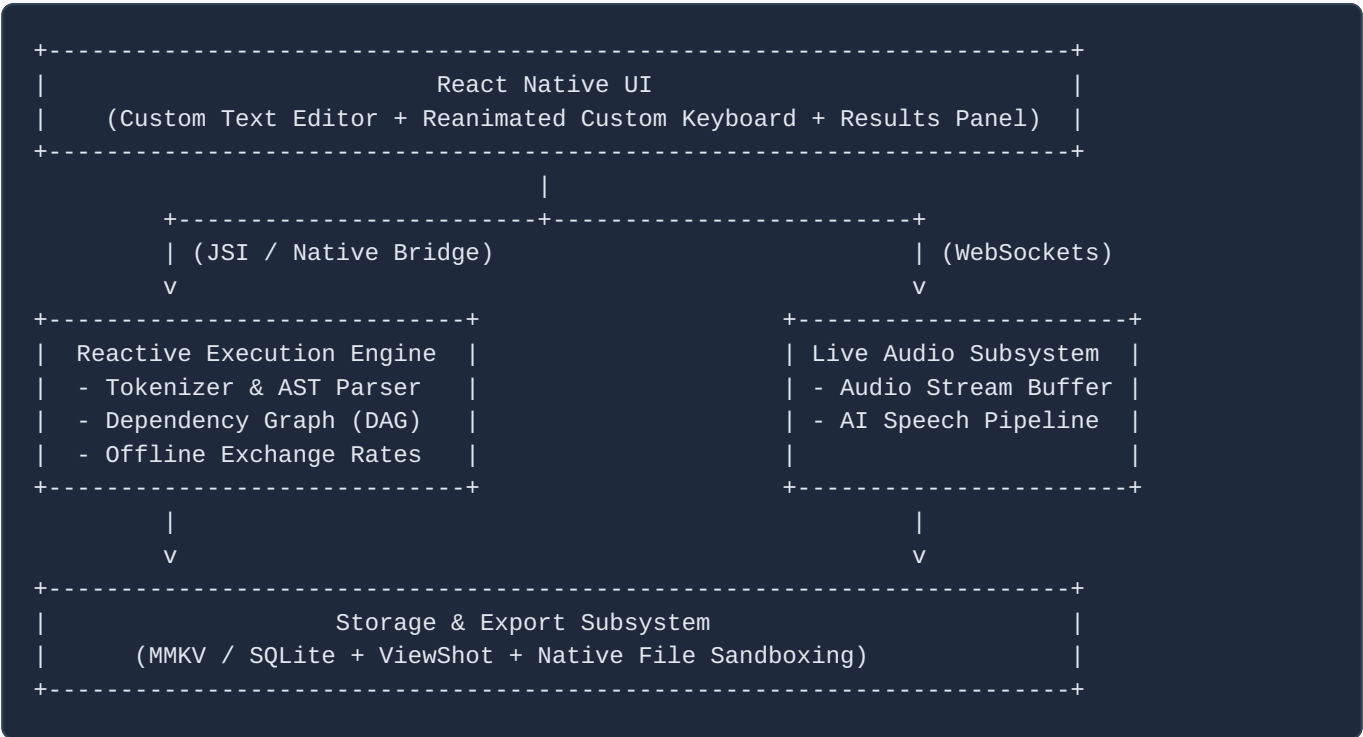
System Documentation

Smart Notepad Calculator (CalcNote Evolution)

This document specifies the architecture, data pipeline, and system integration for a high-performance, real-time notepad calculator built using React Native. The application features live mathematical evaluation, streaming voice transcription, cross-currency calculations, and advanced document exporting.

1. System Architecture Overview

The system relies on a decoupled, reactive architecture where the user interface (UI) thread remains entirely unblocked. Heavy computations—such as text parsing, abstract syntax tree (AST) evaluation, and dependency graph resolution—are offloaded or handled via synchronous native bindings.



2. Core Subsystems & Components

A. The Reactive Execution Engine

The editor evaluates text line-by-line while maintaining inter-line dependencies.

- **Tokenizer/Parser Matrix:** Strips non-mathematical prose using localized Regular Expressions or a formal grammar configuration (e.g., Peggy/Ohm.js). It identifies variables, math operations, and line tokens (e.g., Line 1 or \$1).

- **Dependency Graph:** Modeled using a Directed Acyclic Graph (DAG). When a line changes, only that line and its downstream dependents are re-evaluated, avoiding full-document recalculation loops.
- **Performance Optimization:** For extensive documents, the evaluation engine executes via React Native JSI or an isolated Web Worker thread to eliminate JS bridge serialization bottlenecks.

B. Live Voice Transcription Pipeline

Enables users to dictate calculations hands-free with real-time text injection.

- **Audio Buffer Stream:** Audio is sampled at 16kHz via native microphone hooks and buffered into lightweight Base64 or PCM chunks.
- **Transcription Transport:** Audio chunks stream continuously over a secure WebSocket connection to an AI provider optimized for mathematical dictation.
- **State Injection:** The returned transcription payload injects text directly into the active cursor position, triggering the Execution Engine instantly.

C. Live Currency & Unit Matrix

Handles cross-currency math (e.g., 50 USD + 20 EUR to CAD) entirely offline.

- **Sync Mechanism:** A background service fetches exchange rate payloads every 12 to 24 hours.
- **Data Cache:** Payloads are stored as flat JSON inside MMKV for microsecond-level synchronous retrieval during live typing.

3. Technology Stack & Dependency Mapping

Subsystem	Preferred Core Dependencies	Architectural Purpose
State & Graph	mobx / mobx-react-lite	Manages granular reactive updates for the DAG without full-tree re-renders.
Storage	react-native-mmkv	High-speed, JSI-backed storage for app settings and currency matrices.
Parser Base	mathjs	Handles standard scientific notation, units, and base conversions.
Live Audio		

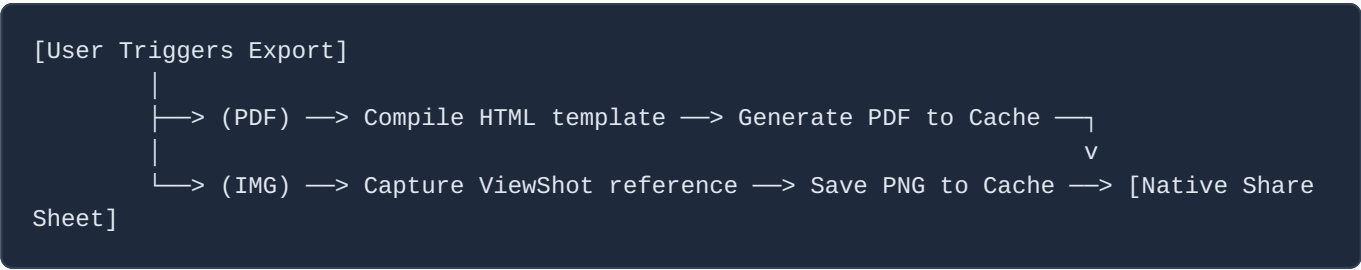
Subsystem	Preferred Core Dependencies	Architectural Purpose
	react-native-live-audio-stream	Non-blocking raw PCM audio capturing for streaming pipelines.
File Export	react-native-view-shot & react-native-html-to-pdf	Generates high-fidelity visual images and printable documents.
Sharing	react-native-share	Resolves cross-platform sandboxing permissions to open native share sheets.

4. End-to-End Data Flows

Live Calculation Sequence

1. The user inputs text or triggers the Voice Pipeline.
2. The Editor emits a string state update.
3. The **Parser** tokenizes the string, looking up offline exchange rates from MMKV if currency syntax is present.
4. The **DAG** updates the specific node representing the active line.
5. Downstream calculations re-evaluate synchronously.
6. The UI updates the line result column.

Document Export & Share Sequence



5. Security & Error Handling Strategies

- **Circular Reference Guard:** The DAG validation loop inspects updates before evaluation. If a circular reference is caught, the engine throws an immediate `#REF!` syntax error to avoid infinite stack overflows.
- **Network Resilience:** If currency API tokens expire or a network connection is lost, the engine falls back gracefully to the most recent cached MMKV exchange rates.
- **Audio Sanitization:** Local audio streams are discarded from the temporary buffer instantly upon transmission to maintain absolute user data privacy.